

Alternative Ways to Do and Verify What We Have Done So Far

A Unified Benchmark of Grokking Predictors in Neural Networks — a stage-by-stage catalogue of cross-checks, from the grokking reproduction up to Predictor 5 (HTSR Alpha)

Prepared for: Jonathan John · Supervisor: Prof. Dr.-Ing. Sheikh Faisal Rashid · 26 September 2026

Based on: `context.md` (full history since 6 July 2026), `06_Code/`, and `08_Experiments/results/nanda_unified/`.

Nothing in the project was changed for this report.

0. What this document is

So far the work has gone in one straight line. We reproduced grokking, built the pipeline, and then implemented five predictors one after another, each measured one way and judged one way. All five failed.

In a colloquium, the natural question after a negative result is: "Did you check it some other way?" For example: "Did you try another definition of grok? Did you verify your NC1 code on a known answer? Did you use a different kernel? Did you check the result with more seeds?"

This document collects, for **every stage of the work done so far**, the other reasonable ways of doing the same thing or verifying it. The purpose is simple. For each alternative, either you run it and have the result, or you know exactly why it was not needed. Nothing here is a new predictor or a new contribution. Every item is either a standard variant of what we did, or an independent cross-check of it.

How each item is marked

Mark	Meaning	Typical time
A	Needs only the saved arrays (<code>.npy</code> , <code>.json</code>). No model needed.	Minutes
B	Needs the saved checkpoints (801 per seed, already on disk and on GitHub). No retraining.	Minutes to a few hours
C	Needs new training runs (about 2.1 hours per seed on MPS).	Hours to days

Items marked ★ are in the "minimum defensible set" in Section 11: the ones I recommend running first, because they are cheap and cover the questions most likely to come.

Useful fact before we start. The pipeline saved a checkpoint every 50 epochs for all 5 seeds. So almost every check below on the four checkpoint predictors (Spectral, AGE, HTSR, Dropout variance) is type A or B. Nothing needs retraining unless the check is about the *training itself* (optimiser, data split, task). This is the biggest advantage the current setup gives you, so use it.

1. Stage 1 — The grokking reproduction itself (M1 gate and the Nanda-Unified substrate)

What we did. Built the transformer from scratch. Reproduced grokking first on $p = 97$ (single head), then moved to four heads, then to the Nanda-Unified protocol ($p = 113$, small initialisation, betas 0.9/0.98). We confirmed grokking by the test-accuracy curve, and by the weight-norm curve matching the shape of Nanda's Figure 7.

The weak point here is this: we have shown that *grokking happens*. We have not yet shown that *our* model groks by the *same mechanism* Nanda describes. All five predictors are judged on this substrate, so this is worth checking

independently.

#	Alternative / cross-check	What it verifies	How	Cost
1.1 ★	Fourier analysis of the token embedding	That the model learns Nanda's Fourier algorithm (a few key frequencies), not some other solution	Take <code>token_embedding.weight[:113]</code> at each checkpoint, apply the DFT over the 113 tokens, plot the norm per frequency. Nanda's Figure 3 shows a handful of key frequencies becoming dominant.	B
1.2 ★	Nanda's restricted and excluded loss	That our run goes through Nanda's three phases (memorisation, circuit formation, cleanup) at the expected times	Using the key frequencies from 1.1, project the logits onto / away from them at each checkpoint, as in Nanda Section 5.1	B
1.3	Gini coefficient of Fourier components (Nanda Figure 7)	Sparsity of the learned solution increasing through training	From the DFT in 1.1	B
1.4	Direct overlay of our $\sum w^2$ on Nanda's Figure 7	Quantitative, not just visual, agreement	Done once for the old <code>nanda_l2_p113/</code> folder; redo for the current 5-seed run with epochs rescaled by grok epoch	A
1.5	Reference implementation cross-check	That our hand-written transformer is not the cause of any difference	Train one seed with TransformerLens <code>HookedTransformer</code> using Nanda's configuration, or load our weights into it and compare logits	C (or B for logit comparison)
1.6	CPU vs MPS, and a same-seed re-run	Numerical reproducibility of grok timing	Re-run one seed on MPS and on CPU; compare grok epochs	C

Why 1.1 and 1.2 matter so much: if they show the Nanda mechanism, you can say "the substrate is not only a grokking model, it is Nanda's grokking model, verified mechanistically". Then no predictor failure can be blamed on a wrong substrate.

2. Stage 2 — Data pipeline and train/test split

What we did. All 113×113 pairs; random 30/70 split with `random_split` after `torch.manual_seed(seed)`; input `[a, b, =]`; label $(a + b) \bmod 113$. 3830 training and 8939 test examples.

#	Alternative / cross-check	What it verifies	How	Cost
2.1 ★	Data sanity tests	No overlap between train and test; correct labels; "=" token is 113; every class present in train	Small script over the rebuilt split per seed; also count training examples per class (AGE skips empty classes, so check none is empty)	A
2.2 ★	Commutativity check	That the model is not simply "memorising the mirror". About 30% of test pairs (a, b) have their mirror (b, a) in the training set	At each checkpoint, split test accuracy into "mirror in train" and "mirror not in train". If both groups jump together at grok, the jump is real generalisation	B

2.3	Separate data-split seed from initialisation seed	Whether the 3.4-fold spread in grok epoch comes from the split or the initialisation	Small factorial design, for example 2 splits × 2 initialisations	C
2.4	Different train fraction (for example 0.4)	That the predictor verdicts do not depend on one particular grok time scale	Retrain with a larger fraction; grok comes earlier, then all checkpoint predictors run as usual	C
2.5	Different operation (for example subtraction)	Task generality	Same protocol, different label function	C

3. Stage 3 — Training protocol

What we did. Full-batch AdamW, lr 1e-3, weight decay 1.0, betas (0.9, 0.98), 40,000 epochs, no warmup, 5 seeds. Brief post-grok accuracy collapses were seen and explained as "consistent with the slingshot mechanism", but not tested.

#	Alternative / cross-check	What it verifies	How	Cost
3.1	Weight decay = 0 control run (one seed)	That grokking here is weight-decay-driven, as Nanda says. Expected result: no grok, or a much later one	One training run	C
3.2	Lower learning rate or weight decay (one seed)	Whether the post-grok collapses disappear, which would support the slingshot explanation	One training run; count dips as before	C
3.3 ★	Check whether the collapses coincide with the predictor events	That the late events (variance peak, NC1 minimum, alignment maximum) are not caused by the dips	Compare dip epochs from <code>test_acc_history</code> with event epochs	A
3.4	Numerical precision check at near-zero loss	That the post-grok spikes are not float32 round-off	Re-evaluate the training loss in float64 at checkpoints around a dip	B
3.5	Longer training (for example 80,000 epochs, one seed)	What happens to the "still falling at the last checkpoint" signals	One run, or read horizon dependence from saved data (see 5.4)	C

4. Stage 4 — Definition of the grok epoch

What we did. Grok epoch = first epoch with test accuracy above 0.9. Also reported at 0.95 and 0.99.

#	Alternative	What it verifies	How	Cost
4.1 ★	Onset thresholds 0.1 and 0.5	That the verdict does not rest on the end of the transition. (Note: earlier grok only makes late predictors look later, so this is a one-sided robustness argument.)	Already computable: onset (0.1) epochs are 11023, 5392, 7925, 8588, 21723	A
4.2 ★	"Sustained" grok	That brief crossings do not define grok. Because of the post-grok dips, "first crossing" and "stays above" can differ	First epoch after which test accuracy stays above 0.9 for, say, 1% of the run	A
4.3	Loss-based grok	Independence from the accuracy metric	Epoch of steepest fall of test loss. Test loss is not saved per epoch, but it can be computed on the checkpoint grid	B
4.4	Midpoint of a sigmoid fitted to test accuracy	A smooth, noise-free grok time	Fit a logistic curve on log-epoch per seed	A

5. Stage 5 — Checkpoint schedule and evaluation grid

What we did. The first design used 24 log-spaced checkpoints (inherited from Dropout variance). This was replaced by saving every 50 epochs (801 per seed) and evaluating on a 100-epoch grid (401 points), derived from two stated requirements.

#	Alternative	What it verifies	How	Cost
5.1 ★	Grid sensitivity: 50 vs 100 vs 250 vs the old 24 points	That no verdict depends on the grid. This was planned on 25 September but not done	--eval_every 50/250 for AGE and HTSR (cheap); Spectral on 250 if time allows	B
5.2	Re-measure transition widths on the new run	That the grid design still holds	Widths from 0.1 to 0.9 are now 1964, 1529, 1585, 2431, 2024 epochs. Narrowest 1529 vs design value 1570, so the design holds. Just record it.	A
5.3	Dense local grid around grok	Exact timing of any signal near the transition	Evaluate every 50 epochs only inside ± 3000 epochs of grok	B
5.4 ★	Horizon truncation	That the events are not artefacts of stopping at 40,000 epochs	Recompute events on checkpoints below 20k and 30k. Quick check already done: events move a lot with horizon, but stay after grok whenever grok is inside the horizon	A

6. Stage 6 — How a predictor is scored

What we did. One event per predictor (the extreme value of its signal, or for L2 Norm our own MA rules). Two tests: (1) the event is before grok in 5/5 seeds; (2) the event epoch tracks grok across seeds (Spearman ρ). The criteria went through three versions (7 Aug, 7 Sep, 26 Sep).

This is the stage where alternatives help the most. All five predictors failed under one event rule, so the obvious question is whether they would fail under other reasonable scoring methods too. The items below are **sensitivity checks on the verdict**, not new detection methods. Present them that way, to stay inside the 25 September scope decision.

#	Alternative scoring	What it verifies	How	Cost
6.1 ★	Grok-normalised time	Whether the signals' features line up with grok at all, without any event rule	Divide each seed's epoch axis by its grok epoch and overlay the five seeds for each signal. If a feature lines up at, say, 0.8 on this axis in all seeds, it tracks grok; if not, it does not	A
6.2 ★	Lagged cross-correlation	Lead or lag without choosing any threshold	Correlate the rate of change of each signal with the rate of change of test accuracy at different lags; report the lag of maximum correlation per seed. Negative lag means the signal moves first	A
6.3 ★	AUC over time (the "Leaderboard: AUC" from the original Gantt plan)	Threshold-free discrimination of pre-grok from post-grok checkpoints	For each checkpoint window before grok, compute the ROC AUC of the signal value for "late pre-grok" vs "early pre-grok" across seeds	A
6.4	Smoothed extreme value	That noise in the flat tail is not choosing the extreme	Take the extreme of a smoothed curve (log-epoch smoothing, as in the L2 work)	A
6.5	Offline change-point detection	Whether the signal has a regime change and when	Standard change-point method on each signal (for example PELT from the <code>ruptures</code> package). Report where the largest change point falls relative to grok	A

6.6	Same event rule for L2 Norm as for the others	That L2 is judged exactly like Predictors 2–5 (its MA rules are pilot-phase rules, not from a paper)	Extreme value of $\sum w^2$ and of its post-memorisation decline	A
6.7	Retrospective fraction-of-change events	Already done on 26 September (10% and 50% levels). Keep it as the documented sensitivity look, labelled illustrative	Nothing new	A (done)
6.8	All three versions of the criteria	That the verdict does not depend on which version of the criteria is used	Apply the 7 August three-criteria version (including the noise-floor criterion) to all five predictors as well	A

Controls for the scoring itself

#	Control	What it verifies	How	Cost
6.9 ★	Positive control: test loss (and optionally Nanda's excluded loss from 1.2)	That the two tests <i>can</i> be passed by a signal that is truly tied to grok. Without this, "everything fails" can be read as "the test is impossible"	Test loss on the checkpoint grid, scored exactly like the predictors	B
6.10 ★	Negative controls	That a meaningless signal fails. Examples: a fixed epoch (the mean grok epoch), "time since memorisation" (memorisation is at epoch 205–213 in every seed), random walk	Score them with the same two tests	A

Statistics

#	Alternative	What it verifies	How	Cost
6.11 ★	Exact tests and leave-one-seed-out	That seed 4 alone does not drive any conclusion	Exact permutation p-values for every ρ ; recompute every correlation with each seed removed; sign test for Test 1 (0/5 has $p = 0.5^5 \approx 0.031$)	A
6.12	Bootstrap confidence intervals on lead times	Uncertainty of the lead / lag numbers	Resample seeds	A
6.13	More seeds (for example 10)	Statistical power for the tracking test	Five more trainings, then all checkpoint predictors	C

7. Stage 7 — Checking each predictor's code on known answers

This is a separate kind of defence. Before arguing about what a predictor means, you should be able to say: "Each implementation was tested on an input where the right answer is known." At present, the predictors are verified by smoke tests (the code runs), not by known-answer tests (the code gives the right number). All the tests below are type A, need no model, and take minutes.

#	Predictor	Known-answer test
7.1 ★	L2 Norm	Compare <code>compute_l2_norm</code> with <code>torch.cat + torch.linalg.norm</code> on a random model; check $\ell_2^2 = \sum w^2$ and that per-module sums add up to the total
7.2 ★	Dropout variance	Build a fake "model" whose example-wise correctness probabilities q_i are known; check the measured variance matches $\frac{1}{n^2} \sum_i q_i(1 - q_i)$. Also check that dropout really switches on in <code>train()</code> and off in <code>eval()</code>
7.3 ★	Spectral (Canatar)	Synthetic Φ where the labels lie exactly in the top eigenvector: expect $k_{90} = 1$ and maximum alignment. Labels orthogonal to Φ : expect no alignment
7.4 ★	AGE (NC1)	The 2-D toy example already worked out on 22 September (answer ≈ 0.125) as a unit test; plus a perfectly collapsed input (NC1 = 0)
7.5 ★	HTSR Alpha	Draw samples from a power law with known exponent (for example $\alpha = 2.5$) and check the fit recovers it; also a random Gaussian matrix, whose spectrum should <i>not</i> look heavy-tailed
7.6	Independent implementation	HTSR: run the <code>weightwatcher</code> package (already installed) on a few checkpoints and compare its α with ours. NC1: a short independent NumPy version. This is the strongest "is your code right?" answer
7.7 ★	Audit script	One script that recomputes every number quoted in the thesis tables from the raw arrays, so no number is hand-copied

Known code issue to fix first: `recompute_from_checkpoints()` rebuilds the test split without re-seeding. It did not affect any current number (dropout variance was computed live), but it must be fixed before any Dropout recompute from checkpoints (items in 8.2).

8. Stage 8 — Alternatives for each predictor

For each predictor: the standard variants of the *same* signal. The source paper's version stays the main result. These variants answer "would a different but reasonable reading of the same paper change your verdict?"

8.1 L2 Norm (Predictor 1)

What we did. Five detection strategies on single-head $p = 97$; then MA-crossover and MA-of-MA zero-crossing on four-head $p = 97$ and on $p = 113$. All fire at nearly the same epoch in every seed.

#	Alternative	Why	Cost
8.1.1 ★	Per-module norms (embedding, attention, MLP, unembedding), already logged every epoch	The total is dominated by different modules at different times. A module-level signal could behave differently	A
8.1.2	Norm relative to initialisation, $\ w_t\ /\ w_0\ $, and distance from initialisation, $\ w_t - w_0\ $	This is the framing of Liu et al. (Omnigrok), the actual source of the weight-norm idea	B
8.1.3	$\sum w^2$ vs $\sqrt{\sum w^2}$	Monotone transform, so the extreme-value epochs are identical. Worth one sentence only	A
8.1.4	Weights only vs weights and biases	MLP biases are included in the norm now	B

8.1.5	The three substrates already tried (single-head $p = 97$, four-head $p = 97$, four-head $p = 113$)	Already done. This is your strongest existing robustness row: same verdict on all three	done
-------	---	---	------

8.2 Dropout (Predictor 2)

What we did. Variance of test accuracy over 100 stochastic passes at rate 0.5 on 401 checkpoints; final-epoch dropout gap at five rates. Variance peak comes after grok in 5/5.

#	Alternative	Why	Cost
8.2.1 ★	Rate 0.3 (the rate of Salah and Yevick's Figure 1), plus 0.1 and 0.7	Removes the "you used a different rate" objection	B
8.2.2 ★	Binomial null comparison	Quick check done: measured variance follows $m(1 - m)/n$ with correlation 0.95–0.98. Make it a proper figure	A
8.2.3	Dropout location: MLP hidden layer (as in Salah and Yevick's MLP) and embeddings	Our dropout acts only on the two residual branch outputs. Hidden-layer dropout can be added at evaluation time with a forward hook on <code>mlp_in</code> , without retraining	B
8.2.4	Measured on the training set as well as the test set	Whether robustness appears on data the model memorised earlier	B
8.2.5	Number of passes: 30 vs 100 (already have both, from v4 and now)	Convergence of the variance estimate	A
8.2.6	Dropout gap at all five rates on every checkpoint, not just the final epoch	Turns the gap into a timed signal; the DRC snapshots on the eval grid are already saved	A

8.3 Spectral — Canatar task-model alignment (Predictor 3)

What we did. Last-layer representation kernel on the 3830 training examples; eigen-decomposition; cumulative task power $C(k)$, k_{90} , alignment score. Extremes land after grok in 5/5.

#	Alternative	Why	Cost
8.3.1 ★	$C(k)$ and k_{90} on the non-zero part of the spectrum only	The kernel has rank at most 128, so the other ~3700 eigenvalues are zero and their order is arbitrary. The current $k_{90} \approx 3400$ is read inside that arbitrary part. This is a correction, not an option	B
8.3.2 ★	Effective rank / participation ratio of the kernel spectrum	Quick check on the saved top 50 eigenvalues: it falls from about 36 to about 9–16 at grok. This replaces the recorded explanation "no rank collapse"	A
8.3.3	Canatar's actual output: predicted generalisation error	Canatar's theory predicts the test error of kernel regression from the spectrum and alignment. Compare that predicted error with the network's real test error over checkpoints. This is the most faithful use of the paper	B
8.3.4	Empirical NTK instead of the last-layer kernel	Canatar's theory is about the NTK. Compute gradient Gram matrices on a subsample (for example 500 training examples) at a coarser grid	B (slow)
8.3.5	Kernel from other layers (embedding output, attention output)	The last layer is not the only reasonable choice	B
8.3.6	Kernel on test inputs	Alignment on unseen data, closer to "generalisation"	B

8.4 AGE — NC1 (Predictor 4)

What we did. $\text{Tr}(\Sigma_W) / \text{Tr}(\Sigma_B)$ with equal class weights, on the last-layer training representations. NC1 falls about 660-fold; its minimum is after grok in 5/5.

#	Alternative	Why	Cost
---	-------------	-----	------

8.4.1 ★	Papayan's exact form $\frac{1}{C} \text{Tr}(\Sigma_W \Sigma_B^\dagger)$	Our trace ratio is a common simplification, not the paper's formula. About 20 seconds per seed	B
8.4.2	The other neural-collapse measures from the same paper (NC2 equiangular means, NC3 self-duality with the classifier, NC4 nearest-class-centre agreement)	Same paper, same checkpoints; shows whether any part of neural collapse leads grok	B
8.4.3	Sample-weighted instead of class-weighted averages	Different but standard weighting	B
8.4.4	NC1 on test representations	Collapse on unseen data is closer to generalisation	B

8.5 HTSR Alpha (Predictor 5)

What we did. ML power-law fit with automatic $x_{\min}$ (`powerlaw` package) on the six linear layers; mean α . Minimum after grok in 5/5; $\alpha < 2$ in 85–94% of fits.

#	Alternative	Why	Cost
8.5.1 ★	Goodness-of-fit: power law vs exponential, log-normal, truncated power law (<code>distribution_compare</code>)	Part of the Clauset–Shalizi–Newman method itself. Shows how often "power law" is actually the preferred description	B
8.5.2 ★	Cross-check with the <code>weightwatcher</code> package on some checkpoints	Independent implementation (see 7.6)	B
8.5.3	Weighted $\hat{\alpha} = \alpha \log_{10} \lambda_{\max}$ (Martin et al. 2021)	Already rebuildable from saved per-layer values, no recompute	A
8.5.4	Per-layer α instead of the mean	One layer could behave differently from the average. Already saved	A
8.5.5	Include embedding and unembedding matrices	Nanda's Fourier structure is clearest in the embedding	B
8.5.6	Minimum tail size (for example at least 10 eigenvalues)	Tests whether the $\alpha < 2$ values come from very short tails	B

9. Stage 9 — The earlier side-studies (July to September)

These are already finished. They can be presented as robustness evidence you *already have*, which is useful when someone asks "did you try another architecture / setting?".

Earlier work	What it already shows	Small addition that would help
Single-head vs four-head (August)	L2 Norm fails on both architectures	None needed; mention as an architecture check
p = 97 vs p = 113, old vs small initialisation, betas 0.999 vs 0.98	L2 fails on all substrates; the weight-norm curve changes as predicted when init and betas change	One table, "same verdict on three substrates"
v4 run (24 checkpoints, k = 30) vs current run (801 checkpoints, k = 100)	Spectral, AGE and Dropout give the same verdicts under two grids and two sample sizes	One table side by side; this is a ready-made grid and sample-size check
Spectral v1 (weight SVD, stable rank) vs v2 (Canatar)	Two different "spectral" readings, both after grok	Mention as "we also tried a weight-spectrum variant"
Shadow LayerNorm experiment	Adding LayerNorm destabilised training under weight decay 1.0	Note that its motivation was based on a misreading (Nanda has no LayerNorm), but the result still justifies the no-LayerNorm choice

10. Things that may look like alternatives but should NOT be done

You should also have a reason for what you deliberately did not do.

- **Tuning thresholds or windows to make a predictor pass.** This goes against the 25 September scope decision. Answer: "the benchmark tests published methods, not tuned versions."
- **Designing an online detection rule.** Out of scope by the same decision. It stays a documented limitation.
- **Changing the substrate again (LayerNorm, other optimisers) for the main result.** This would break comparability between predictors. Robustness rows (items 2.4, 2.5, 3.1) are separate from the main table.
- **Presenting the retrospective 10% / 50% events as a predictor.** They use the future of the curve, so they can only be a descriptive sensitivity look.

11. Recommended minimum set, in order

If time is limited, these items give the widest cover for the least work. None of them needs retraining.

Order	Items	What you can say afterwards	Cost
1	7.1–7.5, 7.7 known-answer tests and audit script; fix the re-seeding bug	"Every predictor implementation passed a known-answer test, and every number in the thesis is recomputed from raw data by one script."	A
2	1.1, 1.2 Fourier analysis and Nanda's progress measures	"The substrate groks by Nanda's mechanism, verified mechanistically, not only by the accuracy curve."	B
3	6.9, 6.10 positive and negative controls	"The scoring can be passed by a signal truly tied to grok, and it rejects meaningless signals."	A/B
4	6.1, 6.2, 6.3 grok-normalised overlays, lagged cross-correlation, AUC	"The verdict holds without any event rule or threshold."	A
5	4.1, 4.2, 5.1, 5.4 grok definitions, grid, horizon	"The verdict holds for other grok definitions, grids and training lengths."	A/B
6	8.3.1, 8.3.2, 8.4.1, 8.5.1, 8.2.1, 8.2.2	"For each predictor I also checked the main alternative reading of the source paper; none changes the verdict." (Only if that is what the results show.)	B
7	6.11 exact tests, leave-one-seed-out	"No conclusion depends on seed 4."	A

Optional, needing retraining, to discuss with Prof. Rashid: weight decay = 0 control (3.1), split vs initialisation factorial (2.3), one different train fraction (2.4), more seeds (6.13).

Two honest cautions. First, these checks can also change a result. If one of them makes a predictor pass, that is a finding to report, not something to hide. Second, items 8.3.1 and 8.3.2 are not optional in my view: the current Spectral explanation ("no rank collapse") is contradicted by the saved eigenvalues, and should be corrected before the thesis is written.

References mentioned

Canatar, Bordelon, Pehlevan (2021), Nature Communications 12, 2914. · Clauset, Shalizi, Newman (2009), SIAM Review 51(4). · Liu, Michaud, Tegmark (2023), Omnigrok, ICLR 2023, arXiv:2210.01117. · Martin, Peng, Mahoney (2021), Nature Communications. · Nanda, Chan, Lieberum, Smith, Steinhardt (2023), Progress Measures for Grokking via Mechanistic Interpretability, ICLR 2023. · Papayan, Han, Donoho (2020), PNAS 117(40). · Salah, Yevick (2025), arXiv:2507.11645. · Thilak et al. (2022), The Slingshot Mechanism.